

Physics-Backed Dynamic Policy Search for Pilot-Production Viability

AIAA-Style Technical Note

Tyler K. B.

Branch: `codex-viability-prototype`

June 2026

Abstract

This note documents the viability work on the `codex-viability-prototype` branch. The workflow now uses physics-backed direct evaluation as the authority for feasibility claims and casts dynamic policy design as a finite-horizon nonlinear optimal control problem. The implemented policy class uses three open-loop epochs. Each epoch sets seven controls: annual intake, retention, UTE, PAA, maximum manning, FLUG quota, and IPUG quota. A Gaussian-process model is used only to propose candidates; every reported result is checked with direct physics. The primary run evaluated 615 three-epoch policies over a 60-phase horizon. No feasible policy was found. The best policy had $\phi = 6.00173$ and required simultaneous relaxation of the total-pilot window, WG RAP, and FL RAP constraints. A local finite-difference diagnostic shows that the binding constraints respond to different controls, and in some cases move in opposing directions. The result does not prove global infeasibility, but it gives a concrete nearest miss and a defensible basis for the next search or requirements trade.

Nomenclature

x_k	Compressed force and training state at phase k , represented by simulator history: total and line pilots, qualification mix, staff counts, upgrade and carryover burden, and RAP shortfalls.
u_k	Policy control vector applied at phase or epoch k : annual intake, retention, UTE, PAA, max manning, FLUG quota, and IPUG quota.
f_k	Physics-backed simulator transition map from x_k and u_k to x_{k+1} .
$g_i(x)$	Constraint function for target pilot inventory, RAP shortfalls, staff counts, or experience requirements. A feasible policy satisfies $g_i(x) \leq 0$ for all enabled constraints.
ϕ	Maximum normalized constraint violation; $\phi \leq 0$ denotes direct-verified feasibility.
RAP	Ready aircrew program requirement proxy used by the viability constraints.
WG, FL, IP	Wingman, flight lead, and instructor pilot qualification categories.
DOE	Design of experiments.
GPR	Gaussian-process regression.

1 Introduction

The earlier viability workflow used constant policies, static DOE studies, surrogate screening, and two-dimensional feasible-envelope plots. That was a necessary first analysis pass, but it was not enough for policy design. A constant policy cannot represent a phased recovery strategy. A surrogate-only screen cannot support final feasibility claims. Static envelope plots also become difficult to use once several policy levers and constraints are active.

This branch addresses those issues directly. The long-horizon evaluator can now use the single-phase physics allocator instead of the pretrained internal MLP. The direct-physics path has been benchmarked and optimized enough to support parallel candidate batches. A new structured dynamic-policy workflow then searches over three open-loop epochs while keeping direct physics as the source of truth.

2 Physics-Backed Viability Baseline

The evaluator now separates two levels of “direct” evaluation. A direct long-horizon viability run bypasses the outer feasibility surrogate and runs `CAFSimulation` for a selected policy. A physics-backed long-horizon run also bypasses the internal MLP phase surrogate and calls the single-phase physics allocator inside each simulated phase. The runs reported here use `model.phase_backend: physics`.

The performance work changed the search strategy. Early direct physics runs were slow enough to make broad search prohibitively expensive. After allocator and event-capacity optimizations, a documented one-year, eight-policy probe improved from 23.8 s with one worker and 5.8 s with eight workers to 4.1 s with one worker and 1.8 s with eight workers. That made direct physics fast enough for DOE, candidate verification, and moderate structured search. The baseline used here is therefore:

Use physics-backed direct evaluation with `--workers` for final viability claims. Use surrogates only for guidance, screening, and candidate proposal.

3 Finite-Horizon Dynamic-Policy Formulation

Dynamic policy design is treated as an open-loop finite-horizon nonlinear optimal control problem. The horizon contains 60 simulation phases. The implemented policy class uses three equal epochs instead of a fully free per-phase schedule. Each epoch holds seven controls constant over its phase block. The resulting decision vector has 21 dimensions:

$$U = [u_1^\top, u_2^\top, u_3^\top]^\top, \quad u_e = [\text{intake}_e \quad \text{retention}_e \quad \text{UTE}_e \quad \text{PAA}_e \quad \text{manning}_e \quad \text{FLUG}_e \quad \text{IPUG}_e]^\top. \quad (1)$$

The simulator defines the nonlinear transition map,

$$x_{k+1} = f_k(x_k, u_{e(k)}), \quad (2)$$

where $e(k)$ maps phase k to one of the three policy epochs. The objective is to minimize the maximum normalized constraint violation,

$$\min_U \phi(U) = \min_U \max_i \frac{g_i(x_{0:N}; U)}{s_i}, \quad (3)$$

where s_i is the configured constraint scale. A policy is not labeled feasible unless direct physics returns $\phi \leq 0$.

4 Implementation Summary

The implementation keeps the dynamic search inside the `src/viability` boundary. The main changes are:

- `src/viability/dynamic_policy.py`: defines epoch-policy schedules, phase-to-epoch mapping, flattened policy feature names, and unit-cube conversion for bounded policy variables.
- `src/viability/evaluator.py`: adds direct evaluation of `EpochPolicySchedule` objects by applying the active epoch policy before each simulator phase.
- `src/viability/dynamic_search.py`: adds Sobol initialization, structured heuristic seeds, GPR fitting, surrogate-guided candidate proposal, direct physics verification, and local finite-difference diagnostics.
- `src/viability/cli.py`: adds `dynamic-search` and `dynamic-diagnose` commands.
- `tests/test_viability_dynamic_policy.py` and `tests/test_viability_dynamic_search.py`: cover epoch mapping, integer rounding, dynamic simulation policy application, artifact contracts, and diagnostic report generation.

Generated run artifacts stay under ignored `outputs/viability/...`; no generated result files are tracked.

5 Search Procedure

The primary search used the physics-backed configuration in `outputs/viability/dynamic_policy_search/config_physics.yaml`. The procedure was:

1. Generate three-epoch schedules from structured heuristic seeds and a Sobol initial design.
2. Evaluate all initial schedules with direct physics using eight workers.
3. Fit a Matern- $\nu = 1.5$ GPR surrogate to direct-physics ϕ values in normalized control space.
4. Score a large Sobol candidate pool with a lower-confidence-bound rule and add one differential-evolution optimum from the surrogate.
5. Direct-verify the top optimizer-proposed schedules with the physics backend.
6. Run finite-difference perturbations around the best direct-verified schedule.

The primary run evaluated 519 initial schedules and 96 optimizer-proposed schedules, for 615 direct physics evaluations total.

6 Results

No feasible policy was found in the tested three-epoch policy class. Table 1 gives the run summary.

Table 1: Primary dynamic-policy search summary.

Quantity	Value
Epoch count	3
Total simulation phases	60
Initial direct evaluations	519
Optimizer direct evaluations	96
Total direct evaluations	615
Feasible policies found	0
Best ϕ	6.00173
Best active constraint	WG RAP

The best observed schedule was constant across all three epochs:

Control	Value
Annual intake	150
Retention rate	0.55
UTE	20.0
PAA	30
Maximum manning percentage	180.0
FLUG quota per phase	2
IPUG quota per phase	0

The best policy satisfied the final total-pilot constraint, but it failed the minimum total-pilot window after the assessment start. It also left WG RAP and FL RAP violations. Table 2 lists the positive violations at this nearest miss.

The active constraint distribution over all direct evaluations was 325 FL RAP, 173 WG RAP, and 117 total-pilot window. The minimum observed value of each core constraint was negative somewhere in the run: -2331 for the total-pilot window, -0.9335 for WG RAP, -0.8856 for FL RAP, and -9.8083 for IP RAP. Thus, the search did find policies that satisfied each core constraint in isolation. It did not find a policy that satisfied the inventory window, WG RAP, and FL RAP constraints at the same time.

Table 2: Positive constraint relaxations required by the best observed policy.

Constraint	Positive violation
Total-pilot window	456 pilots
WG RAP shortfall	6.00173
FL RAP shortfall	2.86928

7 Local Control Authority Diagnostic

A finite-difference diagnostic was run around the best schedule using perturbations equal to 5 percent of each configured control range, with integer controls rounded and perturbations clipped at bounds. This is a local linearization diagnostic only; it is not a controller design or a substitute for direct verification.

The strongest local sensitivities are summarized in Table 3. Negative sensitivity means increasing the control reduces the corresponding violation locally.

Table 3: Dominant local control authority near the best trajectory.

Response	Highest-authority control	Sensitivity
ϕ	Epoch 3 retention rate	-1.778
Total-pilot window	Epoch 2 retention rate	90.91
WG RAP	Epoch 3 retention rate	-1.778
FL RAP	Epoch 2 retention rate	7.701
IP RAP	Epoch 2 retention rate	-2.318

Retention has the clearest local authority over the binding WG RAP miss. The same neighborhood shows competing effects on FL RAP and the total-pilot window. IPUG perturbations improve IP RAP locally but worsen the current objective and FL RAP near this schedule. The diagnostic therefore points to a joint-feasibility problem, not to a single constraint that is impossible to satisfy on its own.

8 Limitations

A 615-evaluation structured search does not prove global infeasibility. It tests one policy class: three open-loop epochs with bounded controls and surrogate-guided candidate proposal. A fully free per-phase schedule would have many more degrees of freedom. That option was deliberately deferred to keep the implementation interpretable and the evidence tractable.

The GPR surrogate is only a candidate generator. Its fitted length scales suggest weak sensitivity in several sampled directions, but those diagnostics are not feasibility evidence. Direct physics remains the source of truth.

9 Recommendations

The recommended next steps are:

1. Keep the physics backend with worker-parallel direct evaluation as the baseline for final claims.
2. Continue with structured open-loop searches before adding an MPC or a broader controls architecture.
3. For additional search, refine around the best policy with asymmetric epoch patterns in the controls that showed local authority.
4. For decision support, start the requirements trade study from the best observed miss: 456 pilots in the total-pilot window, 6.00173 WG RAP, and 2.86928 FL RAP.

10 Conclusion

The viability workflow now has a physics-backed search and diagnostic loop for structured dynamic policies. The implemented dynamic-policy model treats policy design as finite-horizon control while staying close to the existing simulator interfaces. The primary result is negative: 615 direct physics evaluations found no feasible policy in the three-epoch class. The run still produced a clear nearest miss, quantified the required relaxations, and identified local control authority around the binding constraints. The next step is either a focused structured refinement near the best trajectory or a requirements trade study anchored on that miss.

A Commands and Validation

The primary dynamic search was run with the following command:

```
RUN=outputs/viability/dynamic_policy_search/run_3epoch_512_32768_096
venv/bin/python -m src.viability.cli dynamic-search \
  --config outputs/viability/dynamic_policy_search/config_physics.yaml \
  --output-dir ${RUN} \
  --epochs 3 \
  --initial-samples 512 \
  --optimizer-pool-size 32768 \
  --verify-top 96 \
  --workers 8 \
  --checkpoint-every 25
```

The local diagnostic was run with:

```
RUN=outputs/viability/dynamic_policy_search/run_3epoch_512_32768_096
venv/bin/python -m src.viability.cli dynamic-diagnose \
  --config outputs/viability/dynamic_policy_search/config_physics.yaml \
  --evaluations ${RUN}/all_evaluations.parquet \
  --output-dir ${RUN}/diagnostic \
  --epochs 3 \
  --perturbation-fraction 0.05 \
  --workers 8 \
  --checkpoint-every 10
```

The branch validation reported:

- 43 viability tests passed with `venv/bin/python -m unittest`.
- `venv/bin/python -m compileall src/viability viability_dashboard.py` passed.
- `git diff --check` passed.
- Generated outputs under `outputs/viability` remained ignored and untracked.

References

- [1] *Internal Phase Backend Audit and Surrogate Plan*. Repository document: `docs/internal_phase_surrogate_audit_plan.md`.
- [2] *Viability Physics Evaluation Performance Plan*. Repository document: `docs/viability_physics_performance_plan.md`.
- [3] *Dynamic Policy / Finite-Horizon Control Status*. Ignored run artifact: `outputs/viability/dynamic_policy_search/dynamic_control_status.md`.
- [4] *Dynamic Policy / Finite-Horizon Control Search*. Ignored diagnostic artifact: `outputs/viability/dynamic_policy_search/run_3epoch_512_32768_096/diagnostic/dynamic_control_report.md`.